

Server Setup

Overview

The server side of the DarshanEase application is developed using **Node.js**, **Express.js**, and **MongoDB**. The server handles user authentication, business logic, REST API requests, and database operations, ensuring secure and efficient communication between the client and the database.

Prerequisites

Before setting up the server, ensure the following software is installed:

- Node.js (v18 or later)
- npm (Node Package Manager)
- MongoDB (Local or MongoDB Atlas)
- Visual Studio Code
- Git (Optional)

Step 1: Create the Server Folder

Create a folder for the backend server.

```
mkdir backend
```

```
cd backend
```

Initialize the Node.js project.

```
npm init -y
```

Step 2: Install Required Dependencies

Install the required packages.

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
```

Install development dependencies.

```
npm install --save-dev nodemon
```

Step 3: Create the Project Structure

```
backend/
├── config/
│   └── db.js
├── controllers/
│   ├── authController.js
│   ├── templeController.js
│   ├── bookingController.js
│   └── adminController.js
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js
├── models/
│   ├── User.js
│   ├── Temple.js
│   ├── Slot.js
│   ├── Booking.js
│   └── Admin.js
├── routes/
│   ├── authRoutes.js
│   ├── templeRoutes.js
│   ├── bookingRoutes.js
│   └── adminRoutes.js
├── utils/
├── .env
├── server.js
├── package.json
└── node_modules/
```

Step 4: Configure Environment Variables

Create a **.env** file in the backend directory.

PORT=5000

MONGODB_URI=mongodb://localhost:27017/darshanease

JWT_SECRET=your_secret_key

NODE_ENV=development

For MongoDB Atlas, replace MONGODB_URI with your Atlas connection string.

Step 5: Configure Database Connection

Create a file named **config/db.js**.

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log("MongoDB Connected");
  } catch (error) {
    console.error(error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Step 6: Create the Express Server

Create **server.js**.

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const connectDB = require("../config/db");

const app = express();

connectDB();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.send("DarshanEase API is Running...");
});

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

```
});
```

Step 7: Configure package.json

Update the **scripts** section.

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Step 8: Start the Server

Run the development server.

```
npm run dev
```

Example output:

MongoDB Connected

Server running on port 5000

The backend server is now available at:

<http://localhost:5000>

Server Responsibilities

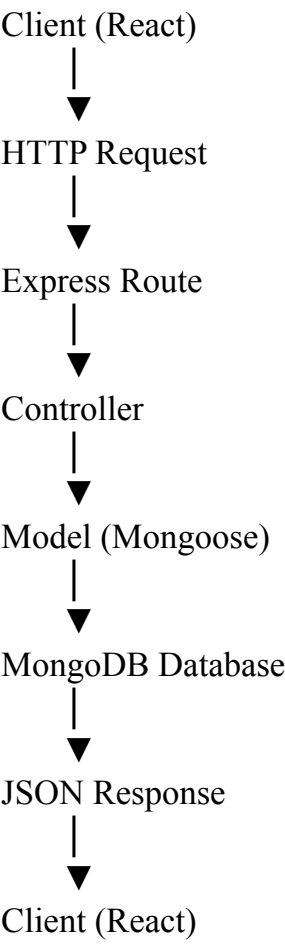
The backend server is responsible for:

- User registration and authentication
- Password encryption using bcrypt
- JWT token generation and verification
- Managing temples and darshan slots
- Processing ticket bookings
- Maintaining booking history
- Validating API requests
- Connecting to MongoDB
- Handling errors and exceptions
- Returning JSON responses to the client

Technologies Used

Technology	Purpose
Node.js	JavaScript Runtime
Express.js	Backend Web Framework
MongoDB	NoSQL Database
Mongoose	MongoDB Object Modeling
JWT	Authentication
bcryptjs	Password Hashing
dotenv	Environment Variable Management
CORS	Cross-Origin Resource Sharing
Nodemon	Automatic Server Restart During Development

Request Flow



Conclusion

The server setup provides a secure and scalable backend for the DarshanEase application. Using Node.js, Express.js, and MongoDB, it manages authentication, business logic, and database operations while exposing RESTful APIs for seamless communication with the React frontend. This architecture supports future enhancements such as online payments, QR code verification, notifications, and mobile application integration.